

2. Introduction

2.1 Project and Team Overview

This section introduces the technical proposal for the **Audio Annotation and Extraction Tool** developed by team **S1_CS_32**, which consists of Aziz Basir, Dzuy Tra, Prachnha Tee, and Vicky Wu.

This report establishes the baseline project feasibility, structural management workflows, and development roadmap for the project

2.2 Background and Problem Space

Audio hobbyists frequently experience highly disjointed and time-consuming collaboration due to a systemic lack of unified production utilities.

Survey data indicates that 80% of creators must manage three or more entirely disconnected applications (such as cloud storage, messaging clients, and email) to share a single track and collect localised feedback.

How many different apps (e.g., Google Drive, Discord, Email) do you use just to share a track and get timestamped feedback?
15 responses

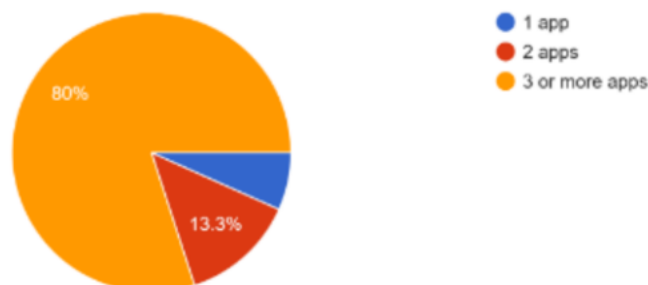


Figure 2.2.1: Survey responses from audio hobbyists regarding application usage for track sharing and feedback collection.

As a result, 86.7% of hobbyists find their current workflows significantly fragmented. This creates an explicit opportunity to design a centralised platform that unifies direct audio timeline playback with precise sample isolation.

How do you feel about your current audio collaboration workflow?

15 responses

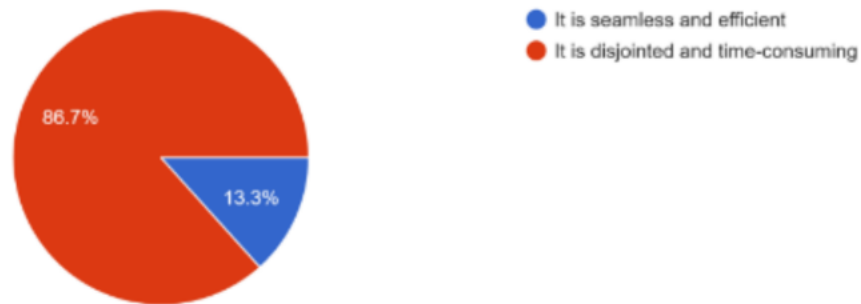


Figure 2.2.2: Survey responses from audio hobbyists evaluating user dissatisfaction and perceived inefficiency within current collaboration workflows.

2.3 Project Aims and Expected Outcomes

The primary project goals are building a centralised web platform for audio hobbyists, ensuring a seamless real-time collaborative user experience, and enabling precise audio annotation and extraction.

The completed application will deliver key functional outcomes, including:

- standard audio playback controls
- an interactive timeline interface to attach labels and specifications at precise timestamps
- a dedicated tool to isolate audio regions for local MP3 or WAV downloads.

The system will enforce secure account management and sustain simultaneous usage for up to 5 concurrent users per file within a strict 2-second synchronisation latency cap.

2.4 Structure of the Report

The proposal is organised into eight chapters:

- **Chapter 1: Front Matter**
 - Provides the table of contents and the team authorship statement
- **Chapter 2: Introduction**
 - Defines the core project identity, problem space context, and target goals
- **Chapter 3: Background Material**
 - Evaluates relevant academic works to establish project rationale
- **Chapter 4: Project Management Plan**
 - Breaks down functional requirements, the iterative Agile development model, schedule milestones, and risk management.
- **Chapter 5: Proposed Methodology**
 - Outlines system components using an Architecture Diagram
- **Chapter 6: Conclusion**
 - Summarises the major technical deliverables and project insights established in the report.
- **Chapter 7: Bibliography**
 - All references in APA 7 format.
- **Chapter 8: Appendices**
 - Includes the RTM, Risk Register, WBS, Project Gantt Chart, and responses to user requirements consultation.

3. Background Material / Literature Review

3.1 Introduction

This section reviews the academic and technical literature underpinning the development of a web-based collaborative audio annotation platform. It examines existing annotation tools and their limitations, the web technologies enabling a browser-based approach, user interface design principles, and the project management methodology adopted by the team.

3.2 Literature Review

3.2.1 Audio Annotation Domains and Uses

Audio annotation is the process of labelling and tagging segments of audio recordings with descriptive data, and plays a foundational role across a range of research and fields of application. In the fields of MIR, annotated music databases are essential for developing computational tools for genre classification, music recommendation and instrument identification (Lesaffre et al., 2004; Zhang et al., 2023). Beyond MIR, precise audio annotation serves as the foundation for machine learning dataset curation, where events must be labelled by start and end times to train reliable auditory models (Flores Carballo et al., 2026). Crowdsourced annotation platforms such as TagATune have further demonstrated that community-driven labelling of audio is both feasible and scalable (Law et al., 2007), and annotation also plays a critical role in wildlife monitoring for passive acoustic species identification (Guo et al., 2024).

3.2.2 Existing Tools and Limitations

Audacity is a free, open-source desktop audio editor widely used for recording, editing, and annotation since its initial release in 2000 (Audacity, n.d.). While it supports timestamp-based label tracks, it is single-user by design. Its cloud syncing via audio.com requires Audacity to be installed, offers only general comments rather than timestamp-specific annotations, and operates as file-based sync rather than true concurrent annotation. Notably, Audacity's interface has been widely criticised for being outdated, with Audacity 4, a major UI overhaul, still unreleased at the time of writing.

Sonic Visualiser is a desktop application for audio visualisation and annotation used in academic music research (Sonic Visualiser, n.d.), but follows a turn-based file-passing collaboration model with no simultaneous multi-user support.

Notetracks is a web-based platform for timestamp-specific feedback targeting musicians and podcasters (Notetracks, n.d.), but restricts sharing to invite-only private links, has no public audio library, and requires a paid subscription, creating a barrier for casual hobbyist users.

Label Studio supports multi-user audio annotation workflows but operates on a task-assignment model rather than concurrent annotation, with a hosted plan starting at \$149 per month, making it inaccessible for hobbyist use (Label Studio, n.d.).

Across all four tools, a consistent gap appears: none provide a free, web-based platform that supports true concurrent annotation of a shared public audio library built for audio hobbyist use.

3.2.3 Underlying Web Technologies

The limitation to desktop-based annotation tools are partly reflected by the technological constraint that existed at the time of their development. Audacity was first released in 2000, predating the emergence of standardised browser-based audio processing capabilities. The Web Audio API, which became a W3C official standard in 2021, enables high-level audio processing and playback directly within the browser without requiring software installation, making a fully web-based annotation platform technically feasible (W3C, 2021). Real-time collaborative functionality is enabled through WebSocket, a protocol that maintains a persistent two-way connection between clients and a server, allowing annotation updates to be broadcasted to all connected users simultaneously (Mozilla Developer Network, 2019). Minimising latency in such interactions is well established, with research demonstrating that user dissatisfaction begins when system response times exceed two seconds, directly informing the project's requirement of a maximum 2-second synchronisation latency for concurrent users (Galletta et al., 2006).

3.2.4 User Interface Design Principles

Nielsen's 10 Usability Heuristics provides a widely adopted framework for UI design, covering principles such as visibility of system status, user control and freedom, error prevention and consistency (Nielsen, 1994). For an audio annotation platform targeting users with varying technical experience, adherence to these heuristics reduces the learning curve and ensures accessibility, directly supporting the project's non-functional usability requirement (R6).

3.2.5 Project Management Methodology

Adaptive project methodology describes the core principle of adjusting to changes regardless of the methodology used, and can incorporate elements from pure Agile frameworks, hybrid approaches or tailored methodologies mixing elements from different sources (Simpson, 2025). Given the nature of this project's evolving requirements and the fixed semester timeline, an adaptive approach was selected over a rigid predictive model. A full justification of the specific adaptations, team roles, and tooling is provided in Chapter 4.

3.3 Rationale

The literature reviewed establishes a clear rationale for the proposed platform. Audio annotation is a well-established practice across multiple domains, but existing tools, whether desktop applications such as Audacity and Sonic Visualiser, or web-based platforms such as Notetracks and Label Studio, fall short of providing a free, accessible and truly concurrent collaborative experience for audio hobbyist use. The development of standardised browser

technologies including the Web Audio API and WebSockets makes a fully web-based approach technically viable. Adherence to Nielsen's usability heuristics ensure the platform remains intuitive, while an adaptive project management methodology provides the flexibility needed for evolving requirements within a fixed timeline. Together, these areas justify the development of a centralised, free, web-based audio annotation platform with a shared public library.

4. Project Management Plan

4.1 Project Scope

The Audio Annotation and Extraction Tool is a web-based platform designed to allow real-time collaboration for musically inclined individuals and hobbyists. The project focuses on delivering core audio interaction capabilities, allowing up to five concurrent users to upload, play, and collaboratively annotate shared audio files. To ensure the project remains achievable and controlled, the product is explicitly limited to a web platform and restricted to standard MP3 and WAV audio formats. Features beyond the project's scope include mobile application development (or PWAs), production-grade deployment (such as DevOps scaling and security hardening), large-scale storage optimisation, and advanced AI audio processing or tagging. Refer to Appendix A for full Requirements Traceability Matrix (RTM)

Functional Requirements (FR):

User Management:

- The system shall require users to create accounts (email, password, username) and securely log in to access the platform's features

File Management:

- The system shall support the uploading and searching of audio files strictly in MP3 and WAV formats

Audio Playback:

- The system shall provide basic audio playback controls, including play, pause, seek, and volume control

Collaborative Annotation:

- Up to 5 users shall be able to annotate the same audio file in real-time. Users can attach specific labels, comments, and specifications to specific audio timestamps, which will be synced and updated in the database.

Audio Extraction:

- Users shall be able to extract specific, selected segments of the audio recording for local download, with all relevant annotations included in the download

File Search:

- Users shall be able to search for audio files in the system.

Non-Functional Requirements (NFR):

Usability:

- The system should maintain a user-friendly and intuitive design, specifically adhering to Jakob Nielsen's 10 Usability Heuristics for User Interface Design

Performance & Concurrency:

- The system should support up to 5 simultaneous users interacting on a single shared audio file, with state synchronisation latency not exceeding 2 seconds

Data Integrity:

- All extracted audio segments must strictly retain their audio quality and format constraints (MP3/WAV) without significant degradation

Requirements were sourced from four origins: the project brief, supervisor consultation, project team decisions and literature research. Requirements R2, R3, and R4 were derived from the project brief. Requirements R1, R5, and R9 were identified through supervisor consultation. Requirements R8, R10, and R6 were determined by the project team, with R6 further supported by literature research. Finally, Requirements R7 was sourced from the project brief, supervisor consultation, and literature research (Galletta et al., 2006).

The Product Acceptance Criteria table can be found in Appendix G.

4.2 Project Organisation

Software Development Life Cycle (SDLC) Model:

The project will use an Adaptive SDLC model. This approach fits the fixed semester timeline and evolving technical requirements, especially real-time collaboration, annotation alignment, and audio extraction. Rather than following a rigid predictive plan, the team will build and test features iteratively through short development cycles. Discord, GitHub, Kanban tracking, and weekly reviews will support communication, version control, and progress monitoring. The specific application of these tools as quality control mechanisms is detailed in Section 4.4.2

Team Organisation, Roles, and Responsibilities:

To ensure clear accountability, each member is assigned a Primary Lead role based on the distinct technical domains mapped out in our Work Breakdown Structure (WBS).

Team members have primary lead roles while sharing implementation, testing, and documentation work. The primary leadership allocations are:

- Aziz leads project management, documentation, WBS, schedule control, and scope management.
- Vicky leads frontend UI/UX and Nielsen heuristic alignment.
- Prachnha leads backend, database, file processing, and real-time synchronisation.
- Dzuy leads QA, integration testing, latency validation, and requirement verification.

4.3 Schedule and Resource Requirements

Overview of Activities and Tasks:

To successfully deliver the Audio Annotation and Extraction Tool within the two-semester timeframe, the project's workload has been divided using a top-down approach. The project activities are grouped by technical domain to allow for development in parallel and clear separation of concerns with the core phases including:

- **Phase 1: Project Management & Planning:** Establishing the foundation, finalising the Requirements Traceability Matrix (RTM), and developing the schedule and WBS.
- **Phase 2: Web Interface:** Designing the UI/UX, user access views, the audio management interface, and the interactive annotation interface.
- **Phase 3: Backend Development:** Setting up database management, user authentication, file management and processing (strictly MP3 and WAV), and the search system.
- **Phase 4: Real-Time Components:** Isolating complex synchronous tasks, including WebSocket implementation and multi-user synchronisation logic to maintain under 2-second latency.
- **Phase 5: Testing:** Conducting continuous integration testing and user testing to validate system boundaries and lossless audio extraction.

A comprehensive breakdown of all specific work-units and deliverables can be found in **Appendix C: Work Breakdown Structure (WBS)**.

Schedule Overview:

The schedule includes task durations, dependencies, milestones, and buffer time for high-risk audio and real-time features. **Appendix C** contains the WBS and **Appendix D** contains the Gantt chart.

Human Resources:

The project will be delivered by a four-member cross-functional team. Each member has a primary leadership role defined in Section 4.2, while implementation, testing, documentation, and review responsibilities are shared across the team. This structure maintains accountability while reducing dependency on any single member.

Technical Resources:

To support the human resources and satisfy the project requirements, the following technical resources will be utilised:

- **Hardware:** Personal development laptops for each team member and a cloud-based remote environment for testing, hosting, and live state synchronisation.
- **Software & Collaboration Tools:**
 - **Communication:** A dedicated Discord server with task-specific channels and a 24-hour reply rule to eliminate task ambiguity.

- **Version Control:** Git and GitHub, enforcing strict branching and pull-request peer reviews to prevent code overwrites and merge conflicts during the development phase.
- **Development:** Standard web development IDEs (VS Code, Cursor), Web Audio APIs for media handling, and WebSocket libraries for real-time concurrency.

4.4 Management Process

4.4.1 Risk Identification, Analysis, and Management Process

To secure project execution integrity and minimise development-related disruptions, we will implement a systematic, four-stage risk management framework.

In accordance with engineering principles, this framework explicitly distinguishes between software development-related risks that occur during active project phases and software product use-related risks that manifest post-deployment, prioritising our tracking efforts entirely on execution lifecycle threats.

The process is structured as follows:

- **Risk Identification:** Potential project threats and positive opportunities are surfaced through recurring team brainstorming sessions and stakeholder requirements consultations. To ensure compliance with unit documentation standards, each identified risk is written using a mandatory cause-and-effect structure:

“Risk of [event impacting objectives] caused by [root cause] resulting in [consequence]”

- **Qualitative and Quantitative Risk Analysis:** Each logged risk scenario is analysed to estimate its relative probability of occurrence and its direct impact on project delivery milestones. In alignment with our spreadsheet template, probability and impact parameters are evaluated quantitatively using a hybrid scale (1, 3, 5, 7, 9) that directly maps to qualitative ranges from *Very low* to *Very high*. An overall priority rating is explicitly calculated for each item using the quantitative formula:

Risk Overall Score = Probability x Impact Score

Risks are dynamically ranked in descending order within the register based on this score, allowing the team to focus active monitoring on top-tier project exposures.

- **Risk Response Strategy Selection:** For every documented threat, the team selects an explicit management approach from the [TARA\(E\)](#) (Appendix F) framework to address the probability or impact before or after an event occurs.
- **Documentation, Tracking, and Active Monitoring:** The complete lifecycle of identified threats is maintained within a Risk Register located in **Appendix B**. Each risk is assigned to a specific team member as the Risk Owner, who tracks development branches for predefined system triggers indicating an active incident.

When a trigger is tripped, the owner instantly executes a predefined **Contingency Plan** to minimise structural project damage. If the initial reaction proves insufficient for critical path milestones, a robust secondary **Fallback Plan** (Plan B) is deployed. The team audits this register weekly during sprint reviews to dynamically transition risk states across *Monitored*, *Triggered*, *Resolved*, or *Lapsed* phases, ensuring that residual exposures and any secondary risks introduced by our chosen strategies are continuously controlled.

4.4.2 Communication plan with stakeholders, Monitoring and Controlling mechanisms

Communication: To support our Adaptive methodology, the following will be used:

- **Internal Team Communication:** Communication will be maintained through regular collaboration and information sharing. Real-time communication platforms, such as Discord, will be used for updates, discussions, and resource sharing to ensure everyone remains informed. Weekly Zoom meetings are dedicated to reviewing progress and assigning tasks.
- **External Stakeholder Communication:** Communication with our unit teaching staff and project supervisors is conducted during scheduled weekly lab sessions. These sessions serve as our formal reporting intervals, utilised to present visual progress, demonstrate functional application prototypes, and obtain feedback on critical project milestones.

Monitoring and Controlling Mechanisms: To maintain project oversight, track development progress, and ensure equal team contributions, the following control mechanisms are actively enforced:

- **Kanban Boards:** Task execution is visually tracked across overlapping sprints in real-time. This allows the team to dynamically monitor workflow states and instantly identify bottlenecks.
- **Meeting & Audits:** Recorded weekly Zoom meeting is documented with time-stamped minutes. These records log "Action Items" with assigned owners and deadlines, ensuring all key decisions are documented.
- **Version Control as a Quality Check:** Version control will serve as a key quality assurance mechanism throughout the project. Git will be used to track all code changes. The commit history provides a clear record of individual team contributions, while peer-reviewed pull requests act as quality checks to prevent untested or conflicting code from merging into the main branch.
- **Live Requirements Traceability Matrix (RTM):** The RTM (Appendix A) will be monitored during reviews to map completed features directly back to our requirements. By continuously monitoring requirement coverage, the project team can identify missing functionality and prevent duplication of work.

5. Methodology

5.1 Required Software and Tools

Frontend: Next.js with React and TypeScript will be used for a unified, type-safe codebase. WaveSurfer.js and the Web Audio API will support waveform display, region selection, and browser playback. Pre-generated waveform peak data will reduce browser memory load.

Backend and Database: Supabase will provide PostgreSQL, Auth, Storage, and Row Level Security. PostgreSQL will store users, projects, audio metadata, annotations, search fields, and version numbers. Supabase Auth will manage secure user sessions, while Supabase Storage will store MP3/WAV source files, waveform data, and extracted segments.

Real-Time Concurrency: Supabase Realtime will provide managed WebSocket-based broadcasting for annotation updates. This removes the need for a custom Socket.io server while still supporting the required under-2-second collaboration latency.

Audio Processing: A separate FFmpeg worker hosted on Railway, Render, or Fly.io will handle metadata extraction, waveform generation, and audio segment extraction. This is required because Vercel serverless functions are unsuitable for long-running, file-heavy audio processing.

5.2 Data Management and Processing

Upload & Validation: MP3/WAV files are uploaded via the Next.js client to Supabase Storage. Strict validation against file extensions, MIME types, and headers prevents unsupported formats.

Metadata & Pre-processing: The FFmpeg worker uses ffprobe to extract metadata (duration, codec, sample rate) and generates compact waveform peak data. The frontend loads this peak data for rapid UI rendering, bypassing the need to decode full audio files in the browser.

Annotation Storage: PostgreSQL stores annotations with fields for file ID, precise timestamps, labels, notes, and version numbers. The original audio files remain entirely unchanged as read-only source files.

5.3 Approach

Real-Time Synchronisation: Annotation edits will use optimistic concurrency control in PostgreSQL, with Supabase Realtime broadcasting committed changes to connected clients. Each annotation record will maintain a version number. When a user submits an edit, the database will accept it only if the submitted version matches the current stored version. This prevents silent overwrites and race conditions. Temporary signals, such as cursor or playback position, will be throttled so persistent annotation updates remain within the two-second latency target.

Lossless Audio Extraction: Extraction will be delegated to the FFmpeg worker. For WAV files, exact boundaries will be calculated using sample positions:

target sample = target time x sample rate

The worker will extract this sample range while preserving original PCM data. For MP3 files, lossless MP3-to-MP3 cutting will be restricted to valid frame boundaries using FFmpeg stream copy. If a user requests an arbitrary cut between MP3 frames, the system will export the segment as WAV/MP3 to preserve exact timing without lossy re-encoding.

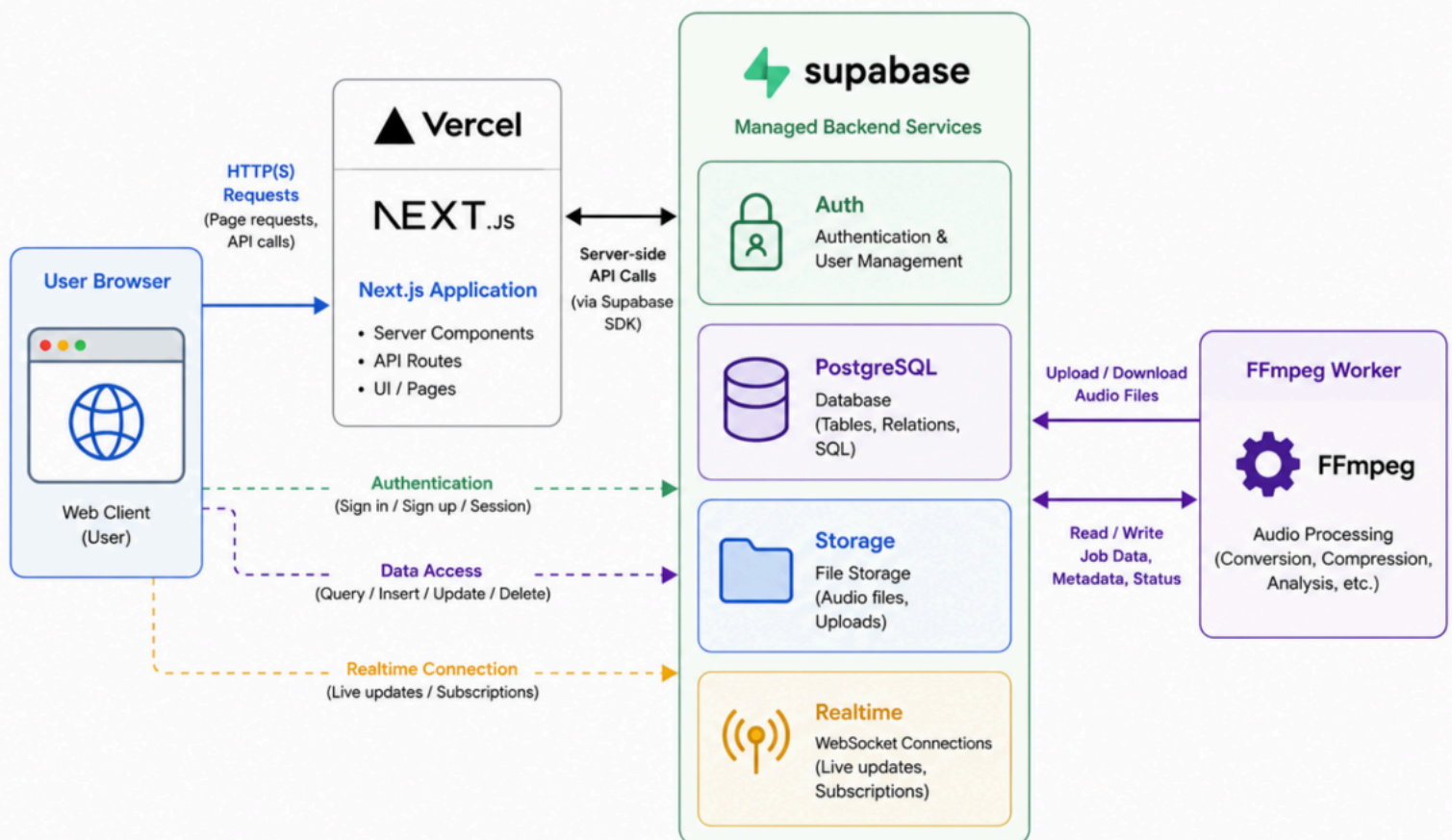
5.4 System Integration and Architecture

The system has four main parts: the Next.js client, Vercel-hosted application, Supabase backend services, and FFmpeg worker. Users interact with the browser interface for login, upload, waveform viewing, annotation, search, and extraction. The browser communicates with Vercel for application routes and Supabase for authentication, storage, database access, and real-time updates.

Supabase Auth controls identity and sessions. Supabase Row Level Security ensures users can access only authorised projects, files, and annotations. Supabase PostgreSQL stores all structured project data. Supabase Storage stores original audio files, waveform data, and extracted segments. Supabase Realtime broadcasts annotation changes to active collaborators.

The FFmpeg worker reads processing jobs from the database, downloads source audio from Supabase Storage, processes it, uploads output files, and updates job status in PostgreSQL.

System Architecture



6. Conclusion

This proposal presents a feasible plan for delivering a web-based Audio Annotation and Extraction Tool for collaborative audio hobbyist workflows. The report defines the project scope, requirements, team roles, schedule, risks, and technical methodology.

The proposed architecture combines a Next.js interface, Supabase backend services, managed real-time updates, and an FFmpeg worker for audio processing. Together, these decisions support the project's main goals: secure access, searchable audio management, real-time annotation, and accurate MP3/WAV segment extraction within the semester timeframe.

7. References

- Audacity. (2019, May 13). *Audacity | free, open source, cross-platform audio software for multi-track recording and editing*. Audacityteam.org. <https://www.audacityteam.org/>
- Audacity. (2024, April 22). *Audacity 3.5 - Cloud Saving, Tempo Detection & Pitch Adjustment*. YouTube. https://www.youtube.com/watch?v=qEAZv_o0HuQ
- Flores Carballo, C. R., Beltrán, J., Pérez, C. B., & Castro, L. A. (2026). Visual tool for audio annotations. *Software Impacts*, 27, 100815. <https://doi.org/10.1016/j.simpa.2026.100815>
- Galletta, D., Henry, R., McCoy, S., & Polak, P. (2006). *Understanding the Direct and Interaction Effects of Web Delay and Related Factors*. ResearchGate; unknown. https://www.researchgate.net/publication/282258873_Understanding_the_Direct_and_Interaction_Effects_of_Web_Delay_and_Related_Factors
- Guo, Q., Duan, S., Xie, J., Dong, X., & Xiao, Z. (2024). Advances in bird sound annotation methods for passive acoustic monitoring. *Biodiversity Science*, 32(10), 24313. <https://doi.org/10.17520/biods.2024313>
- Label Studio – Open Source Data Labeling*. (n.d.). Label Studio. <https://labelstud.io/>
- Law, E., Ahn, L. von, Dannenberg, R. B., & Crawford, M. (2007). TagATune: A Game for Music and Sound Annotation. *ResearchGate*, 361–364. https://www.researchgate.net/publication/220723612_TagATune_A_Game_for_Music_and_Sound_Annotation

- Lesaffre, M., Leman, M., Baets, B. D., & Martens, J.-P. (2004). Methodological Considerations Concerning Manual Annotation Of Musical Audio In Function Of Algorithm Development. *ISMIR 2004, 5th International Conference on Music Information Retrieval, Barcelona, Spain, October 10-14, 2004, Proceedings*. https://www.researchgate.net/publication/220723790_Methodological_Considerations_Concerning_Manual_Annotation_Of_Musical_Audio_In_Function_Of_Algorithm_Development
- Mozilla. (2019, November 28). *The WebSocket API (WebSockets)*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- Nielsen, J. (1994, April 24). *10 Heuristics for User Interface Design*. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Notetracks Pro | Audio Review and Collaboration*. (2024). Notetracks.com. <https://www.notetracks.com/>
- Simpson, L. (2025, September 23). *Agile vs Adaptive Project Management: What's the real difference? | David Simpson Apps*. David Simpson Apps. <https://www.dsapps.dev/blog/agile-vs-adaptive-project-management-difference/>
- Sonic Visualiser*. (n.d.). www.sonicvisualiser.org. <https://www.sonicvisualiser.org/>
- Web Audio API brings audio design to the Web as it becomes a World Wide Web Consortium (W3C) Recommendation*. (2021, June 17). W3C. <https://www.w3.org/press-releases/2021/webaudio/>
- Zhang, R., Wang, H., Liu, L., & Pei, D. (2023). Survey of research on automatic music annotation and classification methods. *Journal of Frontiers of Computer Science and Technology*, 17(6), 1225. <https://doi.org/10.3778/j.issn.1673-9418.2210114>

8. Appendices

Appendix A: Requirements Traceability Matrix (RTM)

Req. ID	Description	Categories	Type(s) (FR/NFR)	Source	Status	Priority
R1	The system shall support uploading files in MP3/WAV formats.	Audio, File management	FR	Supervisor Consultation	Pending	HIGH
R2	The system shall support playback of uploaded audio files and provide basic audio controls (play, pause, seek, volume control etc).	Audio, Playback, UI	FR	Project brief	Pending	HIGH
R3	Users shall be able to annotate audio recording with labels, comments and specifications at specific timestamps.	Annotation, UI	FR	Project brief	Pending	HIGH
R4	Users shall be able to extract selected segments of audio recordings for local download (includes the annotations in download)	Audio, Extraction	FR	Project Brief	Pending	MEDIUM

R5	Users need to create accounts (email, password, username) and log in securely to access annotation features.	User Management , Security	FR	Supervisor Consultation	Pending	HIGH
R6	The system should maintain a user-friendly and intuitive design. The UI must adhere to Jakob Nielsen's 10 Usability Heuristics.	UI, Usability	NFR	Project Team, Literature Research	Pending	MEDIUM
R7	System should support up to 5 users simultaneous usage on a single shared audio file, not exceeding a latency synchronisation of 2 seconds.	Performance, Collaboration	NFR	Project brief, Supervisor Consultation, Literature Research	Pending	HIGH
R8	Extracted Audio Segments should retain quality and format (MP3/WAV)	Performance	NFR	Project Team	Pending	LOW
R9	Users shall be able to search for audio files in the system.	UI, Collaboration	FR	Supervisor Consultation	Pending	MEDIUM

R10	Annotation audio files will be updated in the database and periodically synced.	File management	FR	Project Team	Pending	MEDIUM
-----	---	-----------------	----	--------------	---------	--------

Appendix B: Risk Register

Legend	Numerical
Very low	1
low	3
medium	5
high	7
very high	9

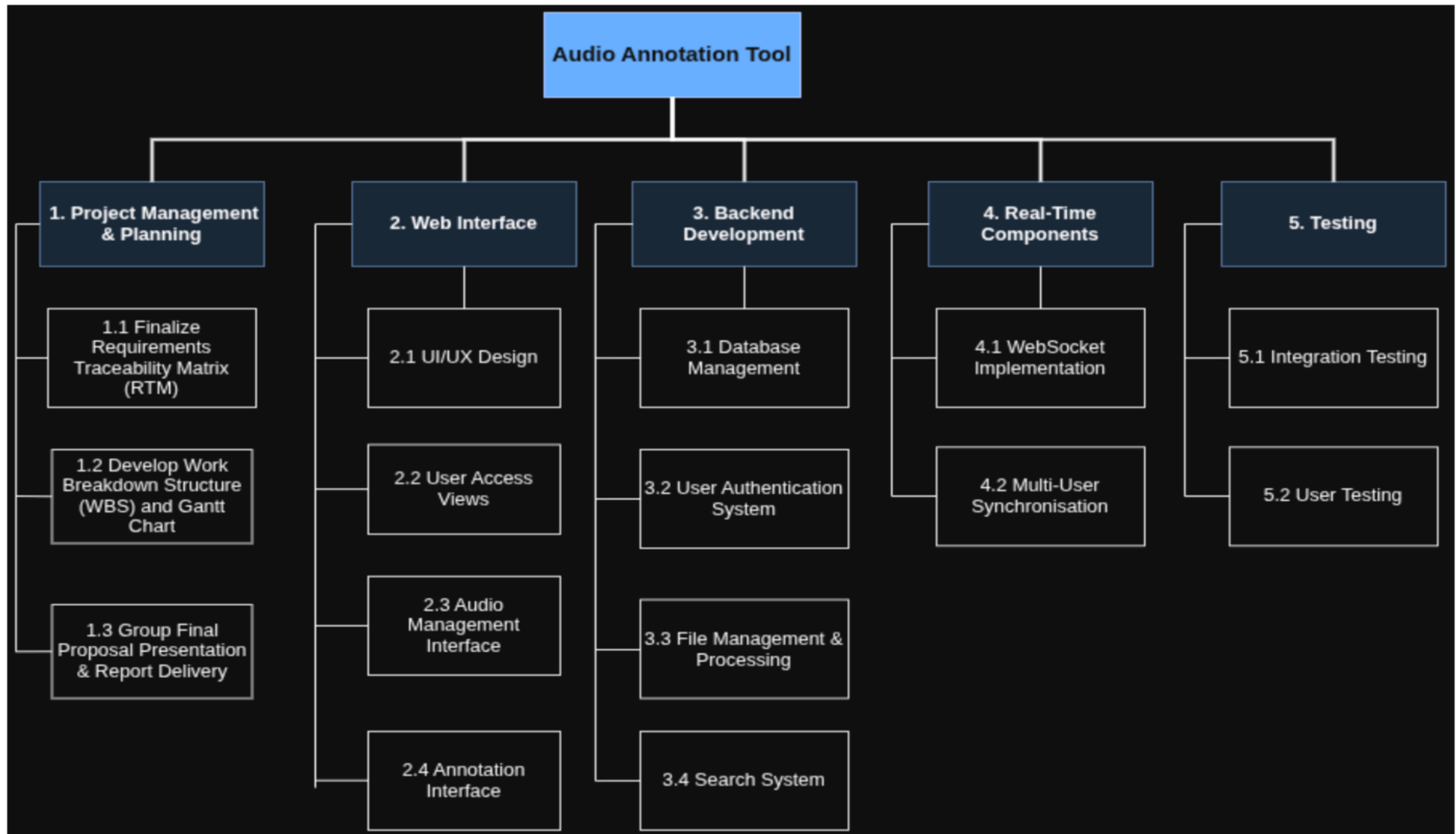
No.	Rank	Risk	Description	Category	Triggers	Root Cause	Potential Responses	Risk Owner	Probability (%)	Impact	Status	Score
R1	1	Real-time Sync Failure	Risk of multi-user synchronisation lag caused by technical complexities in managing concurrent WebSocket broadcasting resulting in exceeding the strict 2-second synchronisation latency cap.	Technical	Synchronisation testing benchmarks consistently cross the 2 second limit during active development sprints.	High message payload broadcast volume over a single server instance during simultaneous multi-user use.	Reduction: Optimise payload sizing and build database write-caching. Fallback: Deploy debounced user interface updates or throttle active connections	Aziz Basir	50%	7	Monitored	350
R2	2	Annotation Misalignment	Risk of visual comment marker misalignment caused by cross-browser rendering or audio sample rate variances resulting in breaking precise timestamp-based annotation alignment targets.	Technical	Automated UI testing logs show comment marker placements displaced by >100ms across different web browsers (Chrome vs. Safari).	Variances in how different web browsers render native HTML5 audio timelines and draw canvas waveforms.	Reduction: Implement standardised audio time-rounding functions based on absolute audio sample counts rather than browser time tracking. Fallback: Introduce manual text-input timestamp correction fields within the user annotation modal.	Vicky Wu	50%	7	Monitored	350

No.	Rank	Risk	Description	Category	Triggers	Root Cause	Potential Responses	Risk Owner	Probability (%)	Impact	Status	Score
R3	3	Duration Misestimation	Risk of sprint schedule slippage caused by unanticipated engineering complexities in simple tasks resulting in delayed milestone deliveries or compressed testing windows.	Project Management	Weekly Kanban board review shows critical-path tasks remaining in the "Blocked" or "In Progress" column past their designated sprint deadline.	Inexperienced team assumptions regarding the baseline implementation overhead of interactive audio-parsing mechanics.	Reduction: Enforce a strict estimation multiplier (e.g., 1.5×) for raw audio handling or real-time networking tasks during sprint planning. Fallback: Descope secondary, low-priority user-requested functional options to protect core release deadlines	Prachnha Tee	70%	5	Monitored	350
R4	4	Audio Extraction Failure	Risk of invalid segment exports caused by errors in client-side slicing logic or serialisation flaws resulting in corrupted local track downloads or missing timeline annotation metadata.	Technical	Testing download cycles yield zero-byte files, broken audio playback, or stripped annotation text strings.	Flaws in the JavaScript client-side MP3/WAV decoding or encoding pipeline array allocations.	Avoidance: Enforce strict array boundary format validation via automated test scripts. Fallback: Archive local output as a ZIP containing a raw file and a clean JSON metadata file.	Vicky Wu	30%	9	Monitored	270
R5	5	Source Merge Conflicts	Risk of source repository branch corruption caused by concurrent development on shared core components without strict branch discipline resulting in regression bugs or broken local deployment builds	Project Management	Continuous Integration (CI) build failures or pull request flags indicating major block conflicts on the repository main branch.	Multiple team members editing the core audio player interface module or database views simultaneously without clear isolation.	Reduction: Implement strict Git branch naming rules and require mandatory peer reviews for any pull requests targeting the main branch. Fallback: Roll back the repository to the last stable, tested milestone release and manually isolate conflicting file deltas.	Dzuy Tra	50%	5	Monitored	250

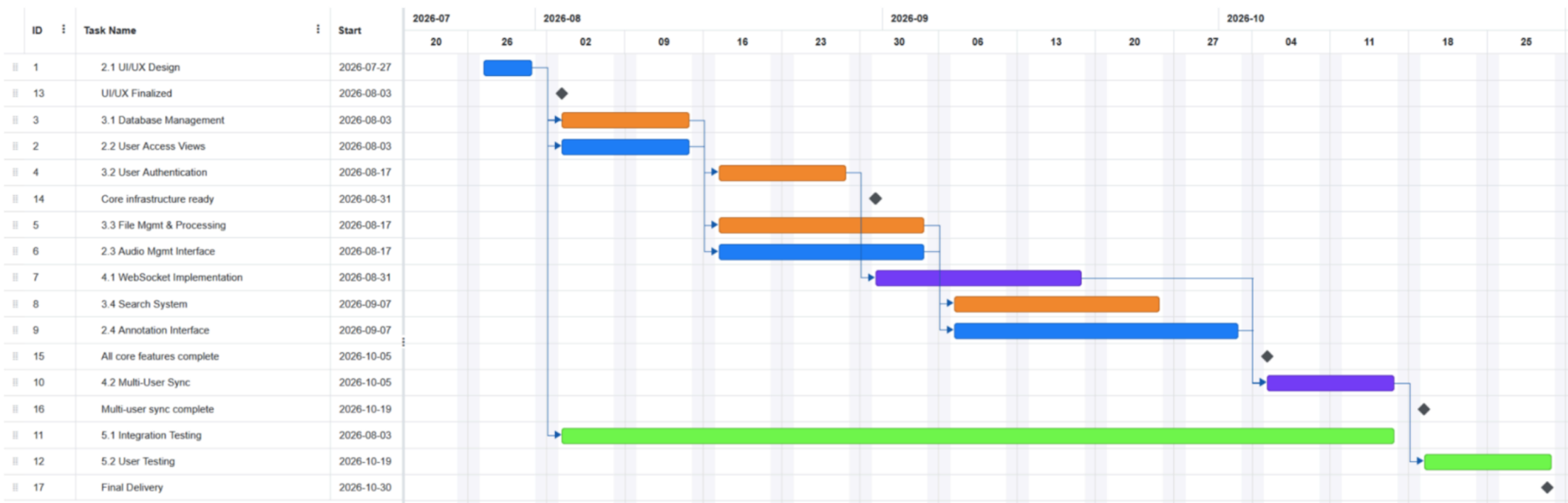
No.	Rank	Risk	Description	Category	Triggers	Root Cause	Potential Responses	Risk Owner	Probability (%)	Impact	Status	Score
R6	6	UI/UX Layout Slippage	Risk of frontend design delays caused by team misalignment on wireframe layouts or component options resulting in missing the schedule baseline and stalling backend infrastructure development.	Project Management	Wireframe assets and interface design assets are not fully finalised within the designated Week 3 milestone timeframe.	Extended team feedback iterations regarding the spatial placement of interactive multi-user annotation components.	Reduction: Shift and redistribute urgent wireframe prototyping tasks across available team members. Fallback: Use basic preset layout components to unblock backend database architecture setups.	Prachnha Tee	50%	5	Monitored	250
R7	7	Critical Resource Loss	Risk of critical path bottlenecks caused by sudden medical emergencies or heavy academic workloads resulting in team member unavailability halting core module development tasks.	Project Management	Dedicated features or code repository branches remain completely inactive for more than 72 consecutive hours.	Unforeseen personal illness or peak end-of-semester assignment constraints forcing project absence.	Reduction: Enforce highly modular development branches via Git version control tools. Fallback: Cross-train team pairs to ensure seamless, collaborative feature code takeovers.	Dzuy Tra	30%	7	Monitored	210
R8	8	Database Concurrency Lag	Risk of system performance degradation caused by database write-locking under concurrent metadata update bursts resulting in transaction response delays that breach real-time performance limits.	Technical	Backend query logs show database transaction response times consistently exceeding 500ms during simulated load testing.	High frequency of simultaneous database write operations when up to 5 concurrent users drops timeline notes on the same file.	Reduction: Implement an in-memory Redis or client-side caching tier to batch incoming metadata before committing to disk. Fallback: Leverage an asynchronous queue framework to handle annotation writes out-of-band from the main WebSocket loop.	Aziz Basir	30%	7	Monitored	210

R9	9	Scope Creep	Risk of project scope creep caused by unclear or evolving stakeholder requirements resulting in sprint schedule slippage or delayed core milestone delivery.	Project Management	Frequent stakeholder requests for new features, repeated changes to user stories, backlog expansion outside sprint planning, or informal acceptance of new requirements without formal approval.	Unclear or evolving requirements	Reduction: Implement strict change control through the Product Manager, ensure all new requirements are logged in the product backlog. Fallback: Reassess sprint priorities, defer or remove lower-priority tasks	Aziz Basir	70%	3	Monitored	210
----	---	-------------	--	--------------------	--	----------------------------------	---	------------	-----	---	-----------	-----

Appendix C: Work Breakdown Structure (WBS)



Appendix D: Gantt Chart



Appendix E: Responses to User Requirements Consultation

	Form_Responses ▾ 				
1	Do you regularly produce music, curate samp ▾	How many different apps (e.g., Google Drive, ▾	How do you feel about your current audio coll ▾	How often do you collect or record long-form ▾	When sourcing samples from long audio files ▾
2	Yes	1 app	It is seamless and efficient	Rarely	Full sections (2+ minutes)
3	Yes	2 apps	It is seamless and efficient	Rarely	Medium blocks (30–60 seconds)
4	Yes	2 apps	It is disjointed and time-consuming	Rarely	Medium blocks (30–60 seconds)
5	Yes	3 or more apps	It is disjointed and time-consuming	Rarely	Medium blocks (30–60 seconds)
6	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
7	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
8	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
9	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
10	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
11	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
12	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
13	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
14	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
15	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)
16	Yes	3 or more apps	It is disjointed and time-consuming	Frequently	Tiny snippets (1–5 seconds)

Appendix F: TARA(E) Framework

- **Transference:** Delegating execution responsibility to external entities, such as offloading user account verification to secure, pre-vetted cryptographic handlers.
- **Avoidance:** Adjusting development scope or engineering practices to entirely eliminate a threat vector, such as restricting system compatibility exclusively to native MP3 and WAV formats to eliminate parsing complexities with other codecs.
- **Reduction (Mitigation):** Executing targeted preventative actions or integration tests during early sprints to actively minimise the likelihood or impact of an incident before it triggers.
- **Acceptance:** Consciously deciding to accept operational consequences without active intervention, reserved exclusively for low-impact threats where active mitigation overhead outpaces project benefits.
- **Escalation:** Routing highly complex procedural or architectural blockages up to senior unit supervisors for higher-level structural resolution.

Appendix G: Product Acceptance Criteria

Requirement ID	Acceptance Criteria
R1	MP3 and WAV files up to 50MB upload successfully and are retrievable for playback
R2	Audio playback controls (play, pause, seek, volume) function correctly
R3	Labels, comments and timestamps are saved and visible to all concurrent users within 2 seconds
R4	Extracted segments download in original MP3/WAV format with annotations included
R5	Users can register and log in securely, with unauthorised access blocked
R6	UI is evaluated against Nielsen's 10 heuristics with no critical violations identified
R7	5 concurrent users can annotate the same file with state changes synchronised within 2 seconds
R8	Extracted audio segments retain original format and show no perceptible quality degradation
R9	Search returns correct audio files matching the query
R10	Annotations are persisted to the database and remain consistent across sessions